Hack the Web

Nishith Rastogi
readersletters@thinkdigit.com

Greasemonkey is an extension for Firefox that permits the end-user to install scripts which makes on-the-fly changes to HTML web page content on the DOMContentLoaded event. This occurs immediately after the page loads in the browser. The changes are executed by various UserScripts, written primarily in JavaScript, which are typically site-specific. The scripts can add features or modify the existing behaviour of the website. There was an addon 'Greasemetal' available for Google Chrome to perform similar functionality, but now its development has ceased as since Chrome 4.0, there is native support for UserScripts. The major advantage of GreaseMonkey being the ability to personalize a web site on the client side, as in even if you are not the author of the site, you can still customise it according to your needs.

To begin you can install the GreaseMonkey addon from https://addons.mozilla.org/en-US/firefox/addon/748/ Once installed you can enable/disable the addon on-the-fly by clicking on the 'Monkey Face' icon in the statusbar. You can find a thousands of UserScripts at http://userscripts.org.

Right-click on the 'Monkey-Face' icon and click on 'New User Script'. For NameSpace you should enter `http://userscripts.org` if you are not planning to host the script on your own web site. The next row 'Includes' is the list of web sites that the script will affect and the 'Excludes' is the exact opposite. After filling in all the details above, click on 'OK', then select the text editor of your choice. At this point you will be presented with a skeleton script that looks like:

```
// ==UserScript==
// @name  Digit_Dev
// @namespace http://userscripts.
org
// @description  For Demonstration
// @include  http://*.facebook.com/*
// ==/UserScript==
```

This is called the metadata information. As you would have guessed you can create the above template manually also from the scratch if you do not appreciate the wizard interface for creating a new script. A small tip would here would be to match both 'http' and https' in your include list. Else the script might not execute over a secure connection.

A simple Hello World script:

```
[AlertBox]
(function()
{
// ==UserScript==
// @name      Digit_Dev
// @namespace http://userscripts.
org
// @description For Demonstration
// @include     *
// ==/UserScript==
alert('Hello Digit Readers');
})();
```

From here on we won't mention metadata anymore. A simple hide image script:

```
var images = document.
getElementsByTagName('img');
for (i=0; i<images.length; i++)
{
  images[i].style.visibility =
'hidden';
}
```

Here, `var images = document.getElementsByTagName('img');` declares a variable called 'images'. Each variable is an object in itself, and has associated members like `var_name.length`, which returns the length of an array. Next the object `document` is the web page itself as described above. From it, by using the member `'getElementsByName'` of the object 'document' we can extract any set of tags from the HTML code. A typical code for insertion of images looks like

```
<img src="Digit.jpg"
alt="important for better google
ranking, also the mouseover text"]
```

Hence the parameter `'img'` for `getElementsByTagName` function.

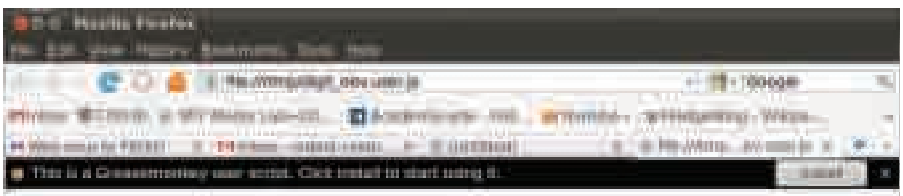
```
for (i=0; i<images.length; i++)
```

This is the very familiar, 'for' syntax. For those who are completely unaware. 'for' is a loop, which executes the statements enclosed in the immediately following paranthesis repeatedly. Here it executes it `i` times, where `i` starts at `0`, goes till `images.length` or the number of images in the document, in an increment of 1 (`i++` is equal to `i=i+1`)

Then `images[i].style.visibility = 'hidden';` Here as explained above images is a array of image, and `images[i]` refers to `i`th image. Each image has a property called style, which has further properties, one of them being `visibility`, which as the name suggests shows/hides an image. The `.` operator in most languages is used to access members of an object class or basically properties of any variable. Of course like above, each of these properties can further have their own properties.

OK, now moving on the next example. Have you ever been irritated by the excessive usage of the words like `Maah` for `My`, `Da` for `The`, or you just want to make the web language a little kid safe or let us say suitable for a living room projector. Then this example of Text Replacement would be perfect for you. This script is by JoeSimmons.

```
// ==UserScript==
// @name           Text Replacement
// @namespace      http://user-
scripts.org/
// @description    Smart text
replacement, will ignore tags
// @include        http://*
// @include        https://*
// ==/UserScript==
// Format: "Word To be Replaced" :
"Replacement Word"
```



Confirmation box ensures you don't install a script by mistake